

CPU Load Optimizer

Static Analysis Tool for Embedded C/H Source Code

Technical Documentation & Architecture Guide

Version 1.0

KSS Platform Team

February 2026

25+ Verified Optimization Rules | No LLM Dependency | Offline Operation

Table of Contents

1. Executive Summary

The **CPU Load Optimizer** is a Python-based static analysis tool designed to identify CPU load optimization opportunities in embedded C and H source code. It was built specifically for the KSS ultrasonic sensor platform project, where measurable CPU load reduction is a primary engineering objective.

The tool operates entirely offline with zero external dependencies beyond the Python 3.8+ standard library. It uses a deterministic, rule-based analysis pipeline (no machine learning or LLM) that produces consistent, repeatable results suitable for corporate deployment and CI/CD integration.

1.1 What the Tool Does

At its core, the tool reads your C/H source files, strips away comments, scans every line against 25+ verified optimization rules, validates each match against its surrounding code context to reduce false positives, and produces an interactive HTML report with per-finding code context, suggested fixes, evidence references, and an estimated CPU load reduction percentage.

1.2 Key Capabilities

- **File Analysis Mode:** Scan individual .c/.h files or entire directories recursively.
- **Git Staged Changes Mode:** Analyze only the lines you are about to commit, filtering out pre-existing tech debt.
- **GUI Launcher:** A tkinter-based graphical interface with dark theme, file/repo browsing, and live status logging.
- **LLM Validation Export:** Generates a compact, token-efficient findings summary for secondary validation by Gemini 3 Pro or similar LLMs.
- **CPU Load Reduction Estimates:** Per-severity and total reduction percentages based on ARM Cortex-M cycle timing data.
- **Interactive HTML Report:** Self-contained, dark-themed dashboard with sortable/filterable findings, code annotations, and bar charts.

2. Architecture Overview

The tool is a single Python file (`cpu_load_optimizer.py`, ~3,800 lines) structured as a pipeline with clearly separated stages. The design philosophy prioritizes simplicity and zero-dependency deployment over framework overhead.

2.1 Pipeline Flow

The analysis pipeline processes source code through four sequential stages:

Step	Stage	Input	Output	Class
1	Preprocessing	Raw source code	Cleaned source + line map	<code>Preprocessor</code>
2	Pattern Matching	Cleaned lines + rules	Raw matches	<code>RulesEngine</code>
3	Context Validation	Raw matches + context	Validated findings	<code>RulesEngine</code>
4	Report Generation	Validated findings	HTML + LLM export	<code>ReportGenerator</code>

2.2 Class Map

The tool consists of 10 classes, each with a single responsibility:

Class	Responsibility
<code>Severity</code>	Enum defining four severity levels (CRITICAL=4, HIGH=3, MEDIUM=2, LOW=1) with string conversion.
<code>Finding</code>	Dataclass holding a single analysis finding: rule ID, location, code context, matched text, recommendation, impact score, and display properties.
<code>Rule</code>	Dataclass defining a single optimization rule: regex pattern, optional validator function, description, evidence references, and impact score.
<code>Preprocessor</code>	Static methods for source code cleanup: comment stripping with line mapping, code context extraction, loop body identification, and function body extraction.
<code>RulesEngine</code>	Core analysis engine containing 25+ Rule definitions and validator functions. Executes all rules against preprocessed source and produces Finding objects.
<code>CodeAnnotator</code>	Optional PIL-based code screenshot generator that creates annotated images showing the issue and suggested fix with arrows.
<code>ReportGenerator</code>	Generates the self-contained interactive HTML report with dashboard, charts, CPU reduction estimates, and expandable finding details.
<code>LLMValidationExport</code>	Generates the compact markdown export package (findings summary + validation prompt + instructions) for secondary LLM review.
<code>GitAnalyzer</code>	Handles git repository operations: detects repos, lists staged files, parses unified diffs for changed line numbers, reads staged file content.
<code>OptimizerGUI</code>	tkinter-based graphical interface with two analysis modes, file/repo browsing, settings panel, and live status logging.

3. Stage 1: Preprocessing

Before any analysis can happen, the raw source code must be cleaned. The `Preprocessor` class handles this with four static methods, each serving a distinct purpose.

3.1 Comment Stripping

Method: `strip_comments(source) → (cleaned, line_map)`

This method removes all C-style (`/* ... */`) and C++-style (`// ...`) comments from the source code. Comments could contain code-like patterns that would trigger false positives (for example, a comment saying `// x / 4 is intentional` would incorrectly match rule C02).

The Line Map: Comment removal can shift line numbers. If a multi-line comment spanning lines 10-15 is removed, line 16 in the original file becomes line 11 in the cleaned version. The line map is a dictionary that maps cleaned line numbers back to original line numbers, ensuring that reported findings always reference the correct position in the original source file.

3.2 Code Context Extraction

Method: `get_context(lines, target_line, window=2) → str`

For each finding, the tool extracts a 5-line window (2 lines above and 2 lines below) centered on the flagged line. The target line is marked with a `>>>` prefix. This context window is displayed in the HTML report so the user can see the finding in its surrounding code without opening the original file.

3.3 Loop Body Identification

Method: `find_loop_bodies(source) → List[(start, end, body)]`

Several rules (C05: Math in Loop, H01: Loop-Invariant Code, H03: strlen in Loop Condition) need to know whether code is inside a loop. This method uses a two-pass approach:

1. **Pass 1 — Find loop keywords:** Scan for lines starting with `for`, `while`, or `do`. Record the line number of each loop header.
2. **Pass 2 — Extract body by brace counting:** Starting from the loop header, count opening and closing braces. When the counter returns to zero, the loop body is complete. The method returns the start line, end line, and the full text of the loop body.

Limitation: This is a lightweight approach based on brace counting, not a full AST parser. It handles nested loops correctly (the brace counter tracks depth) but can be confused by macros that contain unbalanced braces or by conditional compilation blocks. For the typical embedded C code in the KSS project, this approach is reliable and far simpler than integrating a full C parser.

3.4 Function Body Extraction

Method: `find_function_bodies(source) → List[Dict]`

Used by rule H02 (Recursion Detection). This method identifies function definitions by searching for patterns matching `type name(params) {`, then extracts the full function body using the same brace-counting technique. Each function is returned as a dictionary with name, start line, end line, and body text.

4. Stage 2: Pattern Matching (Rules Engine)

The **RulesEngine** is the analytical core of the tool. It contains 25+ optimization rules, each defined as a **Rule** dataclass with a compiled regular expression, metadata, and optional validation logic.

4.1 How Rules are Defined

Each rule is a structured object with the following fields:

Field	Purpose
<code>id</code>	Unique identifier (e.g., C02, H05, L01). Prefix indicates severity class: C=Critical, H=High, M=Medium, L=Low.
<code>pattern</code>	Compiled Python regular expression. This is the primary detection mechanism. If the pattern matches a line, the rule has found a potential issue.
<code>validator</code>	Optional callable function that receives the match object, the line, surrounding lines, and the line index. Returns True if the finding is valid, False to suppress it as a false positive.
<code>needs_context</code>	Boolean flag. If True, the rule is not run line-by-line but is instead run against extracted loop bodies or function bodies for proper structural context.
<code>impact_score</code>	Integer 1-100 indicating the severity of the performance impact if this issue exists in a hot path. Used for sorting findings by priority.
<code>description</code>	Detailed explanation of why this pattern is a problem, including cycle cost information.
<code>evidence</code>	References to industry standards and documentation that validate this rule (MISRA C, ARM manuals, Renesas AppNotes, etc.).
<code>fix_template</code>	Suggested code transformation to resolve the issue.

4.2 Analysis Execution Flow

When the engine runs, it processes rules in three distinct paths depending on the rule type:

Path A: Standard Line-by-Line Rules

Most rules follow this path. The engine iterates over every line of the cleaned source and runs the rule's regex pattern using `finditer()`. For each match, if a validator function exists, it is called. If the validator returns `True` (or no validator exists), a **Finding** object is created with the match details, code context, and original line number (resolved via the line map).

Example (Rule C02 — Division by Power of 2): The regex `(\w+)\s*/\s*(2|4|8|16|32|...)` scans each line. When it matches `raw_adc / 4`, the validator `_validate_not_float_context` checks that `raw_adc` is not a `float` variable. If confirmed as an integer operation, the finding is reported with suggested fix `raw_adc >> 2`.

Path B: Context-Sensitive Rules (Loop-Aware)

Rules marked with `needs_context=True` are not scanned line-by-line. Instead, the engine first extracts all loop bodies using the Preprocessor, then runs the regex only within those loop bodies. This is critical for rules like C05 (Math in Loop) — calling `sqrt()` once at initialization is fine, but calling it inside a `for` loop is expensive.

The line number calculation is adjusted: the match offset within the loop body is added to the loop's start line to determine the original source line.

Path C: Special-Case Detection (Recursion)

Rule H02 (Recursive Functions) cannot be detected by a simple regex because it requires cross-referencing a function's name with calls within its own body. The engine uses `_detect_recursion()`, which iterates over all extracted function definitions and checks if each function's body contains a call to its own name (excluding the definition line itself).

5. Stage 3: Context Validation

Raw regex matching alone would produce many false positives. The validation layer is what separates this tool from a simple `grep`. Each validator function examines the code surrounding a match to decide whether it is a genuine optimization opportunity.

5.1 Validator Functions

Validator	What It Checks & Why
<code>_validate_not_float_context</code>	Used by C02, C03, C04. Scans 20 lines above the match for a direct float/double declaration of the variable being operated on. If the variable is float, the bit-shift optimization does not apply, so the finding is suppressed. Also checks if the current line itself declares a float result. This prevents false positives like flagging <code>raw_adc / 8</code> inside <code>float process_reading(int raw_adc)</code> where <code>raw_adc</code> is actually an <code>int</code> parameter.
<code>_validate_not_standard_func</code>	Used by H04 (Function Calls in Loops). Checks if the function being called is a known standard library function (<code>memcpy</code> , <code>memset</code> , <code>printf</code> , etc.). Standard functions are typically already optimized and should not be flagged as candidates for inlining.
<code>_validate_oversized_type</code>	Used by H05 (Oversized Data Type). Parses the assigned numeric literal and checks if it fits within a <code>uint8_t</code> (0-255). Only values in this range are flagged, because larger values genuinely need wider types.
<code>_validate_missing_const</code>	Used by M01 (Missing const). Checks if the keyword <code>const</code> already appears on the flagged line. Without this, a line like <code>const uint8_t *data</code> would be incorrectly flagged.
<code>_validate_magic_number</code>	Used by L01 (Magic Numbers). Filters out common acceptable literals (0, 1, 2, 10, 100, 1000), values inside <code>#define</code> macros, and values used as array dimensions (e.g., <code>buffer[64]</code>) which are standard practice.

5.2 What the Tool Cannot Validate

Being a static analysis tool without a full C compiler frontend, there are inherent limitations that the user should be aware of:

- **No interprocedural analysis:** The tool cannot trace variable values across function calls. If a variable is modified in another function before the flagged operation, the tool does not know.
- **No runtime frequency knowledge:** The tool cannot know if a flagged loop runs 3 times or 3 million times. That is why CPU reduction estimates are given as ranges (min-max), not exact numbers.
- **No compiler optimization awareness:** At `-O2` or `-O3`, the compiler may already optimize division by power of 2 into a shift. However, many embedded projects use `-O0` or `-O1` for debuggability, where these optimizations are not applied.
- **No macro expansion:** Macros are not expanded before analysis. A macro that hides a division or loop may not be flagged.

- **No pointer aliasing:** The tool cannot determine if two pointers refer to the same memory, which affects the validity of some optimization suggestions.

6. Stage 4: Report Generation

The [ReportGenerator](#) produces a fully self-contained HTML file (no external dependencies) with an interactive dashboard.

6.1 HTML Report Structure

- **Header:** Tool name, analysis timestamp, files analyzed, and mode (Full File or Staged Changes).
- **Stats Grid:** Six metric cards showing total findings and count per severity level (Critical, High, Medium, Low) plus average impact score.
- **CPU Load Reduction Estimation Panel:** Four severity cards showing estimated min-max CPU load reduction percentage per category, with a combined total estimate. See Section 9 for the estimation methodology.
- **Findings by Category Chart:** Horizontal bar chart showing how findings are distributed across categories (Arithmetic, Loop Optimization, Memory, Control Flow, etc.).
- **Top 5 Recommendations:** The five highest-impact findings listed with rule ID, location, and truncated recommendation text.
- **Filter Controls:** Buttons to filter findings by severity level, plus a dropdown to sort by impact score, severity, or line number.
- **Findings Table:** Each finding is a collapsible row. The header shows severity badge, rule ID, rule name, impact score, and file location. Expanding a row reveals: issue description, code context (5-line window with highlighted line), matched code, recommendation, suggested fix, evidence references, and optional annotated screenshot.

6.2 Code Annotations (Optional)

When the `--annotate` flag is used and Pillow is installed, the [CodeAnnotator](#) class renders each finding as a PNG image showing the problematic code with an arrow pointing to the issue and a green box showing the suggested fix. These images are base64-encoded and embedded directly in the HTML, so the report remains a single portable file.

7. Git Staged Changes Analysis

This feature allows you to analyze only the code lines you are about to commit, rather than the entire file. It acts as a pre-commit quality gate, ensuring you do not introduce new CPU load issues without being distracted by pre-existing tech debt in unchanged code.

7.1 How It Works

The staged changes analysis follows a five-step process:

1. **Repository Validation:** The tool calls `git rev-parse --is-inside-work-tree` to confirm the target directory is a git repository, then `git rev-parse --show-toplevel` to find the repository root.
2. **Staged File Discovery:** The tool calls `git diff --cached --name-only --diff-filter=ACM` to list all staged files that were Added, Copied, or Modified. Only .c and .h files are kept.
3. **Changed Line Extraction:** For each staged file, the tool calls `git diff --cached -U0 -- <file>` (with zero context lines) and parses the unified diff hunk headers (`@@ -old +new,count @@`) to build a set of exactly which line numbers in the new version were changed.
4. **Staged Content Reading:** The tool reads the staged version of each file using `git show :<file>` rather than the working copy. This ensures the analysis matches exactly what will be committed, even if the working copy has additional unstaged modifications.
5. **Filtered Analysis:** The full analysis pipeline runs against the staged file content (all 25+ rules). After analysis, findings are filtered: only findings where the `line_number` exists in the changed lines set are kept. Findings on unchanged lines are discarded.

7.2 Why Full-File Analysis with Post-Filtering

An important design decision: the tool analyzes the entire staged file, not just the changed lines in isolation. This is because context matters. A finding at line 50 might depend on a declaration at line 10 (the validator needs to see whether a variable is `float` or `int`). Similarly, C05 (Math in Loop) needs to see the loop boundaries, which may span unchanged lines. Analyzing only the changed lines would miss these relationships and produce incorrect results.

The filtering happens after analysis is complete, preserving analytical accuracy while showing only relevant results.

8. LLM Validation Export

The `--llm-export` flag generates a compact export package in the `Output/llm_validation/` folder designed for secondary validation by an LLM such as Gemini 3 Pro.

8.1 Why Not Upload the HTML Report

The HTML report is designed for human eyes, not LLM consumption. Approximately 70-80% of its token content is CSS, JavaScript, base64-encoded images, and layout markup. Uploading it to an LLM wastes context window on visual formatting data that provides zero analytical value, leaving less room for the LLM to think about your code.

Approach	Tokens Used	LLM Thinking Room	Validation Quality
HTML report + source	~19,000	Low	Poor
LLM export + source	~4,900	Maximum	Best

8.2 Export Package Contents

- **findings_for_review.md:** Every individual finding with its own numbered entry, code context, matched text, and suggested fix. Descriptions and evidence are shown only on the first occurrence of each rule to save tokens; subsequent occurrences reference the earlier description.
- **validation_prompt.md:** A pre-engineered prompt that instructs the LLM to produce a specific verdict (CONFIRMED, FALSE POSITIVE, CONTEXT NEEDED, or PARTIAL) for each finding, with reasoning. It also warns the LLM about known false positive patterns.
- **HOW_TO_VALIDATE.md:** Step-by-step instructions for the end user to run the validation workflow with Gemini 3 Pro.

9. CPU Load Reduction Estimation Model

The tool estimates what percentage of overall CPU load could be reduced if findings in each severity category are resolved. These estimates appear in the HTML report and are based on empirical data from ARM Cortex-M instruction cycle timing tables, Renesas embedded benchmarks, and IEEE Embedded Systems Letters.

9.1 Methodology

Each rule has a base reduction value representing the percentage of total CPU load a single finding could save, assuming the affected code is in a reasonably hot path. For example:

- **C05 (Math in Loop):** 1.5% per finding — functions like `sqrt()` cost 50-200+ cycles per call, multiplied by loop iteration count.
- **C02 (Division by Power of 2):** 0.8% per finding — hardware DIV costs 12-20 cycles vs 1-2 cycles for a right shift.
- **L01 (Magic Number):** 0.05% per finding — minimal direct CPU impact, mainly prevents constant folding optimization.

The per-severity total is calculated by summing all findings' base values, then capping at the severity maximum (CRITICAL: 18%, HIGH: 10%, MEDIUM: 5%, LOW: 2%) to prevent unrealistic estimates.

The final estimate is presented as a **range**: the minimum is 40% of the calculated value (representing cold-path / low-frequency code), and the maximum is 100% (representing hot-path / high-frequency code). The overall total is also capped at 30% maximum.

9.2 Interpretation Guide

A report showing **6.4% — 15.8% total estimated reduction** means: if all 63 findings are in cold paths (rarely executed), the improvement will be around 6.4%. If the findings are in hot paths (main sensor loop, interrupt handlers), the improvement could reach 15.8%. The actual value depends on your code's execution profile, which requires runtime profiling to determine precisely.

10. GUI Interface

The GUI launches when the script is run with no arguments (`python cpu_load_optimizer.py`) or with the `--gui` flag. It uses tkinter, which is included with Python on most platforms, requiring no additional installation.

10.1 Mode 1: Analyze Source File / Directory

The user browses for a specific `.c/.h` file using the Browse File button, or for a directory containing source files using the Browse Folder button. The tool recursively discovers all `.c/.h` files in the selected directory. Settings (severity filter, LLM export, annotations) are configured in the Settings panel, then the Run Analysis button executes the full pipeline.

10.2 Mode 2: Analyze Staged Git Changes

The user browses for a git repository folder. Upon selection, the tool immediately validates it is a git repo and scans for staged `.c/.h` files. A status label shows how many files are staged and how many lines are changed. If no files are staged, a warning instructs the user to run git add first. When analysis runs, only findings on staged lines are reported.

10.3 Post-Analysis

After analysis completes, the GUI shows a summary in the status log (finding counts by severity) and offers to open the HTML report in the default browser. All output files (HTML report, LLM export) are written to the Output/ folder next to the script.

11. Complete Rules Reference

The following table lists all 25+ optimization rules with their severity, category, detection method, and industry evidence.

ID	Rule Name	Category	Impact	Description
C01	Float in Integer Context	Arithmetic	90/100	Flags float/double variables initialized with pure integer literals. On MCUs without FPU, floating-point operations are software-emulated at 10-70x the cost of integer operations.
C02	Division by Power of 2	Arithmetic	85/100	Detects integer division by compile-time constants that are powers of 2. Hardware DIV costs 12-20 cycles; an equivalent right bit-shift costs 1-2 cycles.
C03	Modulo by Power of 2	Arithmetic	85/100	Detects modulo operations by powers of 2. Modulo uses expensive DIV instruction internally; an equivalent bitwise AND ($x \& (N-1)$) is a single cycle.
C04	Multiplication by Power of 2	Arithmetic	75/100	Detects multiplication by powers of 2. MUL costs 3-12 cycles on embedded processors; a left bit-shift costs 1 cycle.
C05	Math Function in Loop	Loop	92/100	Detects calls to sqrt, sin, cos, tan, pow, exp, log inside loop bodies. These functions cost 50-200+ cycles each, multiplied by the loop iteration count.
C06	Dynamic Memory Allocation	Memory	88/100	Flags malloc, calloc, realloc, and free calls. Heap operations are non-deterministic, cause fragmentation, and add ~100-500 cycles of bookkeeping overhead per call.
H01	Loop-Invariant Computation	Loop	68/100	Detects computations inside loop bodies that could be hoisted outside the loop because they do not depend on loop variables.
H02	Recursive Function	Control Flow	65/100	Detects functions that call themselves. Each recursive call adds stack frame overhead (push/pop, branch penalty) and risks stack overflow on resource-constrained MCUs.
H03	strlen() in Loop Condition	Loop	70/100	Detects for($i=0$; $i<\text{strlen}(s)$; $i++$) patterns. strlen() traverses the entire string on each iteration, turning an $O(n)$ loop into $O(n^2)$ complexity.
H04	Function Call in Tight Loop	Loop	55/100	Flags non-standard function calls inside loops. Each call incurs prologue/epilogue overhead (10-20 cycles) that could be eliminated by inlining.
H05	Oversized Data Type	Data Type	60/100	Flags int variables assigned values 0-255, where uint8_t would suffice. On 8/16-bit MCUs, 32-bit operations require multiple instructions.
H06	Float Equality Comparison	Arithmetic	58/100	Detects == or != comparisons on float/double values. Floating-point comparison is slower than integer comparison and unreliable due to rounding.
H07	Struct Pass-by-Value	Memory	62/100	Detects functions that receive struct parameters by value (not pointer). The entire struct is copied to the stack on each call, wasting cycles proportional to struct size.
H08	Redundant Computation	Arithmetic	62/100	Detects identical arithmetic expressions appearing multiple times in the same scope. Each redundant computation wastes CPU cycles that could be saved by computing once and reusing.

M01	Missing const on Pointer	Compiler Hint	45/100	Flags pointer parameters that are not declared const. Without const, the compiler cannot cache the pointed-to value in a register and must reload from memory on each access.
M02	Non-static Internal Function	Compiler Hint	42/100	Flags functions that could be declared static (internal linkage). Static functions enable the compiler to inline them and eliminate dead code.
M03	Global Variable in Loop	Loop	48/100	Detects access to global or extern variables inside loops. Globals must be reloaded from memory on each iteration; a local copy allows register allocation.
M05	Deeply Nested Loops	Loop	50/100	Flags more than 3 levels of nested loops. Deep nesting suggests algorithmic complexity that may benefit from restructuring.
M06	Switch Without Default	Control Flow	35/100	Flags switch statements without a default case. The default case helps the compiler generate an optimal jump table.
M07	Signed Bit-Field	Data Type	30/100	Flags bit-fields declared as int (signed). Signed bit-fields have implementation-defined behavior (MISRA C Rule 6.1) and incur sign-extension overhead.
M08	Sequential Type Casts	Data Type	32/100	Detects chains of type casts. Each cast may generate a conversion instruction; chained casts multiply this overhead.
L01	Magic Number	Maintainability	20/100	Flags numeric literals used directly in code. Named constants (#define) enable constant folding across translation units.
L02	Long If-Else Chain	Control Flow	25/100	Flags 5+ sequential if-else branches. Sequential comparison is $O(n)$; a switch statement enables $O(1)$ jump table lookup.
L03	Computation in Array Index	Arithmetic	18/100	Detects arithmetic expressions used as array indices. Pre-computing the index allows the compiler to select optimal addressing modes.
L04	Missing volatile	Hardware	22/100	Flags direct hardware register access (pointer cast to fixed address) without the volatile qualifier. Missing volatile can cause the compiler to cache stale values.
L05	Uninitialized Variable	Maintainability	15/100	Flags variable declarations without initializers. Uninitialized variables risk undefined behavior and prevent the compiler from optimizing the declaration.

12. Usage Reference

12.1 CLI Commands

The tool supports three modes of operation from the command line:

GUI Mode (No Arguments)

```
python cpu_load_optimizer.py
python cpu_load_optimizer.py --gui
```

File Analysis Mode

```
python cpu_load_optimizer.py src/sensor.c
python cpu_load_optimizer.py ./platform/ -s critical --llm-export
python cpu_load_optimizer.py . --annotate --verbose
```

Staged Changes Mode

```
python cpu_load_optimizer.py --staged /path/to/repo
python cpu_load_optimizer.py --staged . -s high --llm-export -v
```

12.2 CLI Options Reference

Option	Description
<code>target</code>	Path to .c/.h file or directory (positional argument, not used in --staged mode).
<code>--gui</code>	Launch the graphical interface. Also triggered when no arguments are given.
<code>--staged REPO_PATH</code>	Analyze only staged git changes in the given repository path.
<code>-o, --output PATH</code>	Output HTML report path. Default: Output/cpu_load_report.html (or cpu_load_staged_report.html for staged mode).
<code>-s, --severity LEVEL</code>	Minimum severity to include: critical, high, medium, or low (default: low).
<code>--annotate</code>	Enable code screenshot annotations in the report (requires Pillow library).
<code>--llm-export</code>	Generate LLM-optimized validation export files in Output/llm_validation/ folder.
<code>-v, --verbose</code>	Print all findings to the console during analysis.

12.3 Output Files

All output files are written to the Output/ folder next to the script (created automatically if it does not exist):

- `Output/cpu_load_report.html` — Interactive HTML report (file analysis mode).
- `Output/cpu_load_staged_report.html` — Interactive HTML report (staged changes mode).

- [Output/llm_validation/findings_for_review.md](#) — Compact findings summary for LLM review.
- [Output/llm_validation/validation_prompt.md](#) — Pre-built prompt to paste into Gemini/GPT.
- [Output/llm_validation/HOW_TO_VALIDATE.md](#) — Step-by-step instructions for the validation workflow.

13. Industry Evidence & References

Every optimization rule in the tool is backed by one or more of the following industry-standard references:

1. **MISRA C:2012 Guidelines** — Motor Industry Software Reliability Association. The automotive industry's definitive C coding standard, with specific rules on type usage, memory management, and control flow that directly map to the tool's rules.
2. **Renesas Embedded C Programming III Application Note** — Contains detailed instruction cycle counts for Renesas MCU families, including direct comparisons of MUL vs SHIFT, DIV vs SHIFT, and FPU software emulation overhead.
3. **ARM Architecture Reference Manual** — Instruction timing tables for Cortex-M0/M3/M4/M7 cores, covering pipeline behavior, branch prediction, and per-instruction cycle costs.
4. **Embedded.com Engineering Embedded Software Guide** — Practical optimization techniques for resource-constrained embedded systems, including lookup table strategies and loop optimization case studies.
5. **SEI CERT C Coding Standard** — Carnegie Mellon's Software Engineering Institute guidelines for secure and reliable C code, covering uninitialized variables, type safety, and memory management.
6. **GCC Optimization Documentation** — Documentation of GCC's optimization passes, explaining which transformations are applied at each optimization level (-O0 through -O3) and which require source-level changes.
7. **Aho, Sethi, Ullman — Compilers: Principles, Techniques, and Tools (Dragon Book)** — Foundational compiler theory covering common subexpression elimination, loop-invariant code motion, and strength reduction optimizations.
8. **Patterson & Hennessy — Computer Organization and Design** — Hardware architecture fundamentals including memory hierarchy, cache behavior, and instruction pipelining that underpin the cycle cost estimates.

This document was generated for the KSS Platform Team. The CPU Load Optimizer tool is designed for internal use in embedded systems development and is not intended as a replacement for runtime profiling or compiler-level optimization analysis.